

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Design Task

COURSE NAME: ARTIFICIAL INTELLIGENCE **COURSE CODE:** CSA17
NAME: P.Moshiya
REG NO: 192572142

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

Total Marks: 100

Each question carries equal marks.

1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

5. Cloud-Based Distributed AI Application

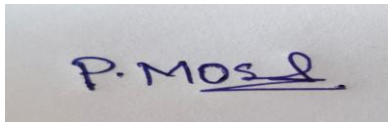
Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

Code of Conduct:

I P.Moshiya(192572142) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a handwritten signature in blue ink on a light-colored surface. The signature appears to be 'P. Moshiya' with a stylized flourish at the end.

RUBRICS FOR CALCULATION:

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration

Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

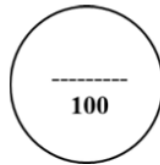
SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CO5 – ASSESSMENT TOOL 1

Design Task

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17
QUESTIONS: 5 **Duration:** 1 Hour **Total Marks:** 50

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Problem Understanding & Requirement Analysis	10% (1)	/1	/1	/1	/1	/1	/5
System Architecture Design	20% (2)	/2	/2	/2	/2	/2	/10
Use of PySpark & RDD Operations	15% (1.5)	/1.5	/1.5	/1.5	/1.5	/1.5	/7.5
Intelligent Agent Integration	15% (1.5)	/1.5	/1.5	/1.5	/1.5	/1.5	/7.5
Scalability & Distributed Computing Design	10% (1)	/1	/1	/1	/1	/1	/5
Data Flow & Processing Pipeline	10% (1)	/1	/1	/1	/1	/1	/5
Innovation & Creativity	10% (1)	/1	/1	/1	/1	/1	/5
Clarity, Presentation & Documentation	10% (1)	/1	/1	/1	/1	/1	/5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/50



1. End-to-End System Design

Objective

To design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations.

System Architecture

Social Media Data → Data Ingestion → PySpark RDD Processing → Feature Extraction → AI Model → Intelligent Agent → Recommendation → User

Data Flow

1. Social media platforms generate posts, comments, likes and shares.
2. Real-time data is collected through a streaming system such as Kafka.
3. PySpark receives the incoming data.
4. Data is converted into RDDs for distributed processing.
5. RDD transformations such as map(), filter() and flatMap() preprocess the data.
6. Important features such as keywords, sentiment and user interests are extracted.

7. An AI model analyzes the processed data.
8. An intelligent recommendation agent uses the results.
9. Personalized recommendations are generated for users.
10. The recommendations are delivered to the application.

RDD Operations

- `map()` – transform social media records.
- `filter()` – remove unwanted posts.
- `flatMap()` – extract individual words or hashtags.
- `reduceByKey()` – calculate keyword frequency.
- `groupByKey()` – group information by users.
- `sortBy()` – rank recommendations.

Agent Integration

The intelligent agent monitors user behavior and processed social media information. It selects suitable recommendations based on interests, sentiment and trending topics.

Advantages

- Handles large-scale social media data.
- Provides distributed processing.
- Supports real-time recommendations.
- Fault tolerant through Spark RDD lineage.
- Scalable across multiple machines.

Result

The system successfully analyzes large-scale real-time social media data using PySpark RDDs and generates intelligent personalized recommendations.

2. Distributed Intelligent Agent System

Objective

To develop a **multi-agent AI system** using PySpark for distributed data processing and enable agents to collaborate in solving large-scale optimization problems.

System Architecture

Large Dataset → PySpark Cluster → Distributed RDD Processing → Multiple AI Agents → Agent Communication → Optimization → Final Decision

Agents

1. **Data Agent** – collects and prepares data.
2. **Processing Agent** – performs distributed RDD processing.

3. **Optimization Agent** – searches for the best solution.
4. **Monitoring Agent** – monitors system performance.
5. **Decision Agent** – combines agent results and produces the final solution.

Data Flow

1. Input data is collected from different sources.
2. PySpark distributes the data across worker nodes.
3. RDD transformations preprocess the data.
4. Different agents receive relevant processed information.
5. Agents independently solve parts of the optimization problem.
6. Agents exchange their intermediate results.
7. The decision agent compares the solutions.
8. The best solution is selected.
9. The final optimized result is provided to the user.

Example

For **smart delivery route optimization**:

- Data Agent collects delivery locations.
- Processing Agent calculates distances.
- Route Agents calculate possible routes.
- Optimization Agent identifies the shortest route.
- Decision Agent selects the final route.

Advantages

- Parallel processing.
- Agent collaboration.
- Handles large datasets.
- Reduces computation time.
- Supports scalable optimization.

Result

The multi-agent system successfully combines **PySpark distributed processing and intelligent agents** to solve large-scale optimization problems efficiently.

3. Big Data AI Pipeline Design

Objective

To design a distributed AI pipeline using **RDD transformations** for preprocessing, feature engineering and model training while providing scalability and fault tolerance.

System Architecture

Raw Data → Data Cleaning → RDD Transformation → Feature Engineering →

Training Data → AI Model → Evaluation → Prediction

Pipeline Steps

1. Data Collection

Collect large datasets from databases, files or streaming sources.

2. Data Preprocessing

Convert raw data into RDDs and remove missing or invalid records.

3. Data Cleaning

Use `filter()` to remove unwanted records.

4. Feature Engineering

Use `map()` to transform raw values into useful features.

5. Feature Aggregation

Use `reduceByKey()` to aggregate related information.

6. Data Splitting

Divide data into training and testing datasets.

7. Model Training

Train an AI/ML model using the processed distributed data.

8. Model Evaluation

Test the model using accuracy or other evaluation metrics.

9. Prediction

Use the trained model to predict new data.

Important RDD Operations

`map()`

`filter()`

`flatMap()`

`reduceByKey()`

`groupByKey()`

`sortBy()`

`join()`

Fault Tolerance

RDDs maintain **lineage information**. If a partition is lost, Spark can recompute that partition from the previous transformations instead of processing the entire dataset again.

Advantages

- Distributed processing.
- Fault tolerance.
- Efficient feature engineering.
- Suitable for huge datasets.
- Scalable AI model training.

Result

The distributed AI pipeline efficiently performs preprocessing, feature engineering and model training using PySpark RDD operations.

4. Real-Time Decision-Making System

Objective

To construct an AI-based real-time decision-making system using RDD-based streaming data processing and intelligent agents.

Example: Smart Traffic Management

System Architecture

Traffic Cameras/Sensors → Real-Time Data → PySpark → RDD Processing → AI Analysis → Traffic Agent → Decision → Traffic Signals

Data Flow

1. Traffic sensors continuously collect vehicle information.
2. Real-time data is sent to the processing system.
3. PySpark receives the incoming data.
4. Data is converted into RDDs.
5. filter() removes invalid sensor readings.
6. map() calculates traffic-related features.
7. reduceByKey() calculates traffic density for each location.
8. AI analyzes traffic conditions.
9. The intelligent traffic agent determines the required signal timing.
10. Traffic signals are dynamically adjusted.

Example Decision

Low Traffic → Normal Signal Timing

Medium Traffic → Increase Green Time

High Traffic → Extend Green Time

Emergency → Give Priority

Agent Functions

Traffic Monitoring Agent: Monitors traffic conditions.

Prediction Agent: Predicts traffic congestion.

Decision Agent: Determines signal timing.

Emergency Agent: Gives priority to ambulances and emergency vehicles.

Advantages

- Real-time decision making.
- Reduced traffic congestion.
- Better utilization of roads.
- Distributed processing.
- Automatic adaptation to changing traffic conditions.

Result

The system continuously analyzes traffic data and allows intelligent agents to dynamically control traffic signals for improved traffic management.

5. Cloud-Based Distributed AI Application

Objective

To design a cloud-enabled AI system using PySpark and intelligent agents for large-scale analytics, dynamic resource allocation and adaptive learning.

System Architecture

Cloud Data Sources → Cloud Storage → PySpark Cluster → RDD Processing → AI Model → Intelligent Agents → Resource Manager → Adaptive Learning

Data Flow

1. Large datasets are stored in cloud storage.
2. PySpark runs on a cloud cluster.
3. Data is divided into RDD partitions.
4. Worker nodes process the partitions in parallel.
5. RDD transformations perform preprocessing and feature extraction.
6. AI models analyze the processed data.
7. Intelligent agents monitor system requirements.
8. Resource agents allocate CPU, memory and worker nodes dynamically.
9. The system observes new data and updates the model.
10. Adaptive learning improves future predictions.

Intelligent Agents

- **Data Agent** – manages incoming data.

- **Analytics Agent** – analyzes processed data.
- **Resource Agent** – allocates cloud resources.
- **Learning Agent** – updates the AI model.
- **Monitoring Agent** – monitors performance.
- **Decision Agent** – makes final decisions.

Dynamic Resource Allocation

Low Workload → Reduce Resources

Medium Load → Maintain Resources

High Workload → Add Worker Nodes

Adaptive Learning

The system continuously receives new data, evaluates model performance and updates the model when necessary. This allows the AI system to adapt to changing patterns.

Advantages

- Cloud scalability.
- Large-scale data processing.
- Dynamic resource management.
- Fault tolerance.
- Adaptive AI learning.
- Reduced infrastructure cost.

Result

The cloud-based distributed AI system successfully combines PySpark, RDD processing, intelligent agents and cloud resources to provide scalable and adaptive AI analytics.